TRAVAUX PRATIQUES : WORD2VEC

Apprentissage et analyse d’embeddings lexicaux sur des résumés de films et des avis Amazon

Rapport de traitement automatique du langage naturel
Année universitaire 2026–2027
Can Pekgoz

Résumé

Ce travail met en œuvre Word2Vec sur deux corpus anglophones de domaines distincts : des résumés de films issus de The Movies Dataset et des avis Amazon concernant les téléphones et leurs accessoires. Après nettoyage et tokenisation, les architectures CBOW et Skip-gram ont été entraînées dans Gensim. Quatre modèles principaux sont étudiés en termes de coût d’entraînement, de vocabulaire, de similarité cosinus et de voisinages lexicaux. Les représentations sont également visualisées par ACP/PCA, puis la sensibilité de trois hyperparamètres est analysée sur des échantillons. Les résultats soulignent un compromis de temps favorable à CBOW et des variations sémantiques dépendant du corpus et du réglage retenu.

1. Introduction et objectifs

Les approches de plongement lexical (word embeddings) représentent les mots par des vecteurs numériques appris à partir de textes. Le présent TP vise à comprendre la logique de Word2Vec et à comparer deux architectures sur deux jeux de données. La consigne porte sur le chargement, le nettoyage, la tokenisation, l’entraînement avec Gensim et l’évaluation des représentations obtenues. La comparaison des temps d’exécution, des similarités et des hyperparamètres constitue une extension expérimentale de ces objectifs.

2. Corpus et préparation des données

Le corpus Films provient de la colonne « overview » du fichier movies_metadata.csv de The Movies Dataset (Kaggle). Le corpus Amazon correspond au champ « reviewText » de fichiers JSON Lines concernant les téléphones et accessoires. Le script accepte différents noms et formats, notamment des fichiers .json et .json.gz, avec détection du format GZIP. La version exacte de l’archive Amazon utilisée n’est pas attestée par les scripts seuls ; il convient de ne pas assimiler automatiquement ce fichier à l’archive SNAP mentionnée dans la consigne.

Tableau 1 — Caractéristiques des corpus après prétraitement

Corpus	Source textuelle	Textes conservés	Vocabulaire (min_count=5)
Films	Résumés « overview »	44 471	22 475
Amazon	Avis « reviewText »	982 041	32 764

Le prétraitement est implémenté avec gensim.utils.simple_preprocess(str(texte), deacc=True, min_len=2). Il transforme le texte en minuscules, retire la plupart des symboles non alphabétiques, supprime les diacritiques et les découpe en tokens.
Les documents contenant moins de trois tokens sont écartés.
Les deux corpus tokenisés sont ensuite sauvegardés en JSON.
Aucune lemmatisation, correction orthographique systématique ou suppression dédiée des mots vides n'est effectuée. Les fautes de frappe et certaines formes proches peuvent donc demeurer dans l'espace vectoriel.

3. Principes théoriques

Word2Vec apprend un vecteur dense pour chaque terme suffisamment fréquent en exploitant les mots présents dans son contexte. Deux variantes sont distinguées : CBOW (Continuous Bag of Words), qui prédit un mot cible à partir des mots voisins, et Skip-gram, qui prédit les mots voisins à partir du mot central.
CBOW correspond à sg=0 et Skip-gram à sg=1 dans Gensim.
Les deux modèles s'appuient sur des contextes locaux plutôt que sur un dictionnaire de synonymes préexistant.
La proximité entre deux mots est ici mesurée par cosinus : $\cos(u,v) = (u \cdot v) / (||u|| \times ||v||)$.
Cette mesure renseigne sur l'orientation des vecteurs et non sur une équivalence de sens stricte. Deux mots peuvent être proches parce qu'ils apparaissent dans des contextes semblables ou dans le même thème, même s'ils ne sont pas substituables l'un à l'autre.

4. Protocole expérimental principal

Le script 02_train_word2vec.py entraîne successivement CBOW et Skip-gram pour chacun des corpus, avec vector_size=100, window=5, min_count=5, epochs=5, seed=42 et un maximum de quatre workers. Les modèles sont sauvegardés localement, tandis que les métriques d'entraînement sont exportées dans training_results.csv.

Les mesures reflètent l'exécution du projet et ne constituent pas un benchmark contrôlé sur des machines différentes.

Tableau 2 — Comparaison des modèles entraînés

Corpus	Architecture	Dimensions	Fenêtre	Époques	Vocabulaire	Durée (s)
Films	CBOW	100	5	5	22 475	6,40
Films	Skip-gram	100	5	5	22 475	19,13
Amazon	CBOW	100	5	5	32 764	140,01
Amazon	Skip-gram	100	5	5	32 764	426,09

Sur les films, Skip-gram nécessite 19,13 / 6,40 = 2,99 fois la durée de CBOW. Sur Amazon, le rapport est 426,09 / 140,01 = 3,04.
Le temps absolu est nettement supérieur pour les avis Amazon, corpus beaucoup plus volumineux. Cependant, l'architecture et le volume ne sont pas les seuls déterminants : le nombre de tokens, les fréquences lexicales et l'environnement d'exécution peuvent également intervenir.

5. Évaluation par similarité cosinus

Pour les résumés de films, les paires évaluées sont king–queen, love–romance, war–battle, murder–killer et family–mother.
Pour Amazon, les paires sont phone–smartphone, battery–charge, screen–display, camera–picture et charger–cable.
Les résultats ci-dessous proviennent de results/similarites.csv, calculé sur les modèles principaux et non sur les sous-échantillons utilisés ensuite.

Tableau 3 — Similarités obtenues sur les films

Paire	CBOW	Skip-gram
king – queen	0,7371	0,7588
love – romance	0,6219	0,6554
war – battle	0,6140	0,5072
murder – killer	0,6750	0,5516
family – mother	0,6227	0,4317

Sur ce jeu de cinq paires, Skip-gram donne une similarité légèrement supérieure pour king–queen et love–romance, tandis que CBOW donne une similarité supérieure pour les trois autres paires.

Cette variation interdit de conclure que l'une des architectures serait systématiquement plus performante sur tous les concepts des synopsis.

Tableau 4 — Similarités obtenues sur Amazon

Paire	CBOW	Skip-gram
phone – smartphone	0,4841	0,5594
battery – charge	0,4246	0,5659
screen – display	0,5338	0,6233
camera – picture	0,2551	0,3976
charger – cable	0,6141	0,6975

Sur Amazon, Skip-gram donne un score plus élevé pour les cinq paires sélectionnées. Cela suggère une structuration contextuelle plus marquée pour ces relations techniques, au prix d'un temps d'entraînement environ trois fois plus élevé.

Il ne s'agit toutefois pas d'une validation externe : la qualité ne peut pas être résumée par la maximisation arbitraire de cinq cosinus.

6. Analyse des voisinages lexicaux

Le fichier mots_similaires.csv regroupe les dix voisins les plus proches de mots cibles. Pour Films/CBOW, « king » est associé notamment à « emperor » (0,7467), « queen » (0,7371), « minister » (0,7283), « president » (0,7137) et « prince » (0,7091).

Pour « murder », les voisins « crime » (0,7346), « suicide » (0,7319), « murderer » (0,7303) et « killing » (0,7185) montrent des liens thématiques cohérents.

Sur Amazon, la proximité entre « case » et « cover » ainsi qu'entre « charger » et « chargers » répond aux usages courants dans des avis techniques.
Des erreurs d'orthographe proches des mots cibles apparaissent également parmi certains voisins, reflétant le bruit des données collectées auprès des utilisateurs.

7. Projection des représentations en deux dimensions

Les graphiques qui suivent proviennent du dépôt GitHub et ont été produits par le script 04_beautiful_graphs.py. Une analyse en composantes principales (PCA) projette les vecteurs d'origine dans un plan 2D.

Les points proches dans la figure ne reproduisent qu'imparfaitement les distances d'origine : l'ACP est avant tout une illustration qualitative des voisinages appris.

Figure 1 — PCA Films, CBOW

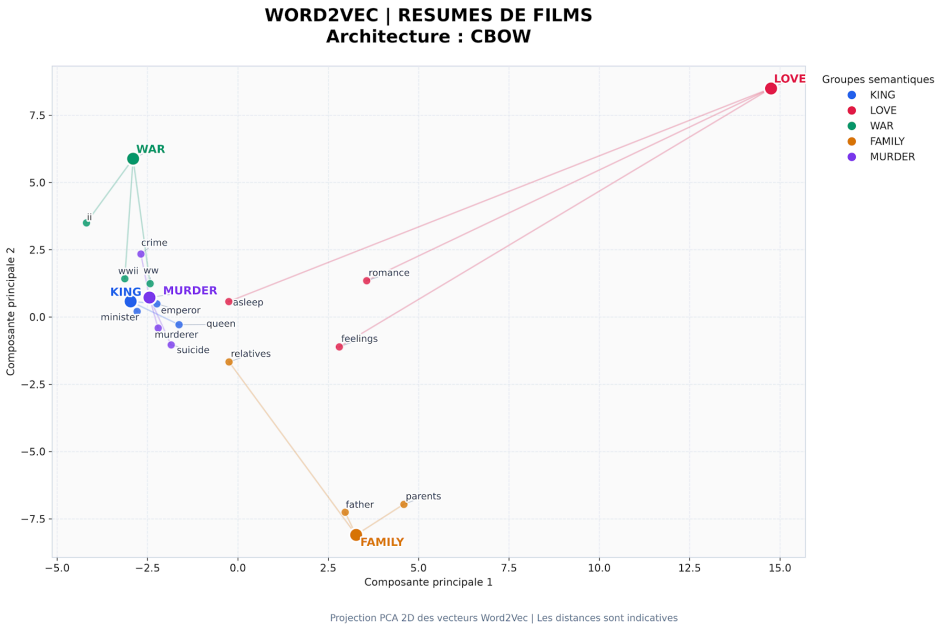


Figure 2 — PCA Films, Skip-gram

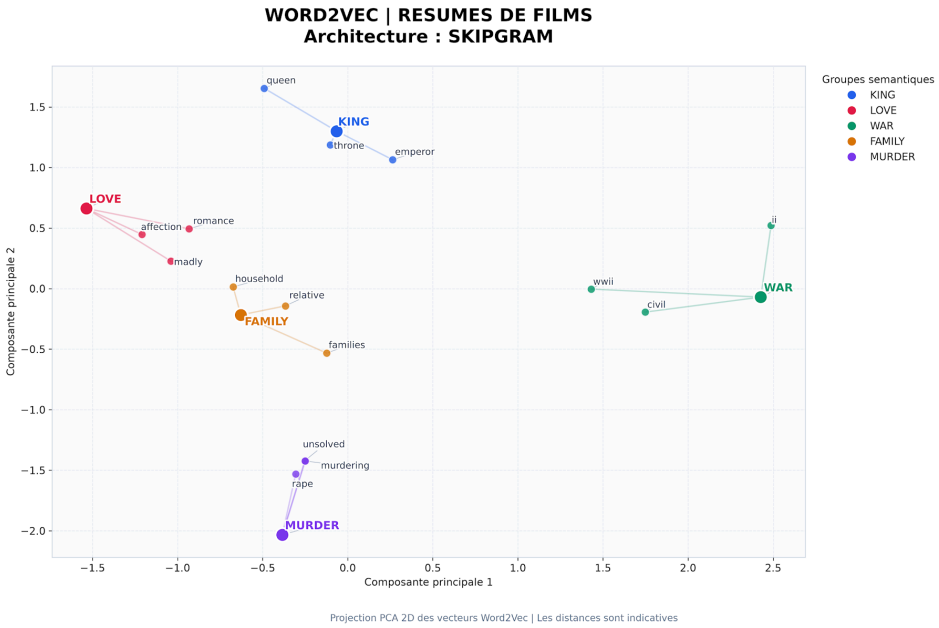


Figure 3 — PCA Amazon, CBOW

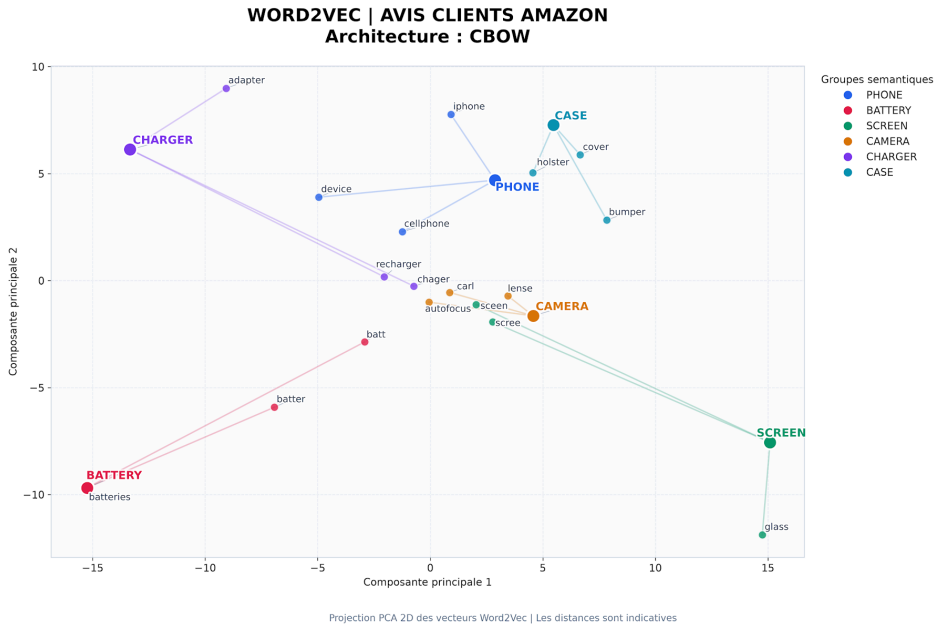
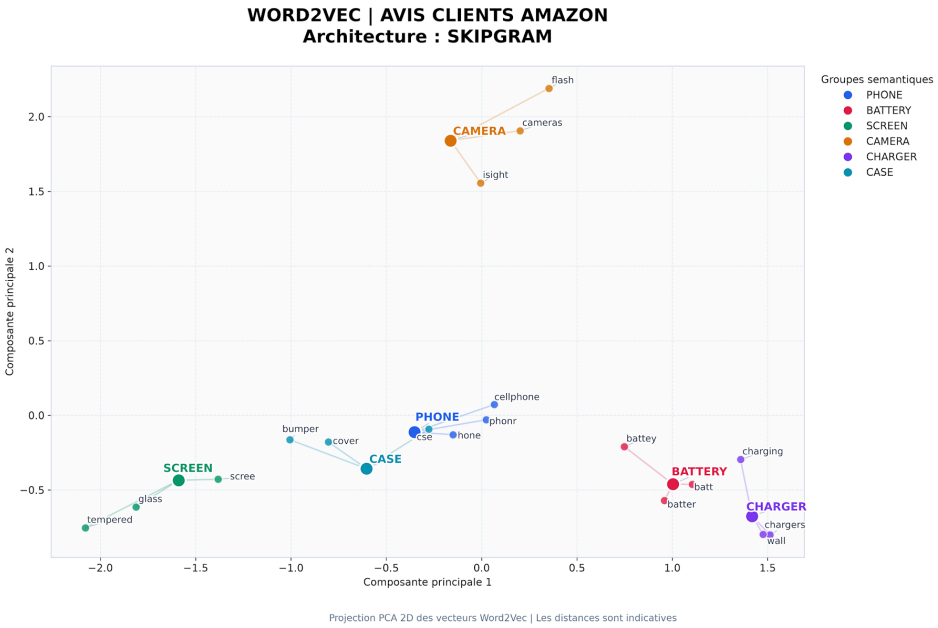


Figure 4 — PCA Amazon, Skip-gram



8. Sensibilité aux hyperparamètres

Le script 05_hyperparameters.py effectue 18 entraînements CBOW supplémentaires : neuf configurations sur 10 000 résumés et neuf sur 30 000 avis Amazon, extraits par échantillonnage aléatoire avec une graine fixe.

Les réglages de base sont vector_size=100, window=5, epochs=5 et min_count=3. Un seul hyperparamètre varie à chaque essai : dimensions (50, 100, 200), fenêtre (2, 5, 10) ou époques (3, 5, 10). Les modèles des expériences sont distincts des quatre modèles principaux.

Tableau 5 — Paramètres et plages testées

Paramètre	Référence	Valeurs évaluées
vector_size (dimensions)	100	50 / 100 / 200
window (contexte)	5	2 / 5 / 10
epochs (passages)	5	3 / 5 / 10
Architecture	CBOW	Fixée
min_count	3	Fixé

Tableau 6 — Effet de window sur Amazon : valeurs représentatives

Paire	window=2	window=5	window=10
battery – charge	0,4181	0,4465	0,5367
screen – display	0,5504	0,5106	0,4997
camera – picture	0,1142	0,1990	0,2496
charger – cable	0,6209	0,6351	0,6839

L’augmentation de la fenêtre améliore le cosinus de battery–charge, camera–picture et charger–cable, mais le diminue pour screen–display.
Une fenêtre plus large peut favoriser les relations thématiques au détriment de certaines proximités lexicales fines.

Aucun paramètre n’est donc optimal de manière universelle sur ces seules quatre paires.

Figure 5 — Films : dimensions vectorielles

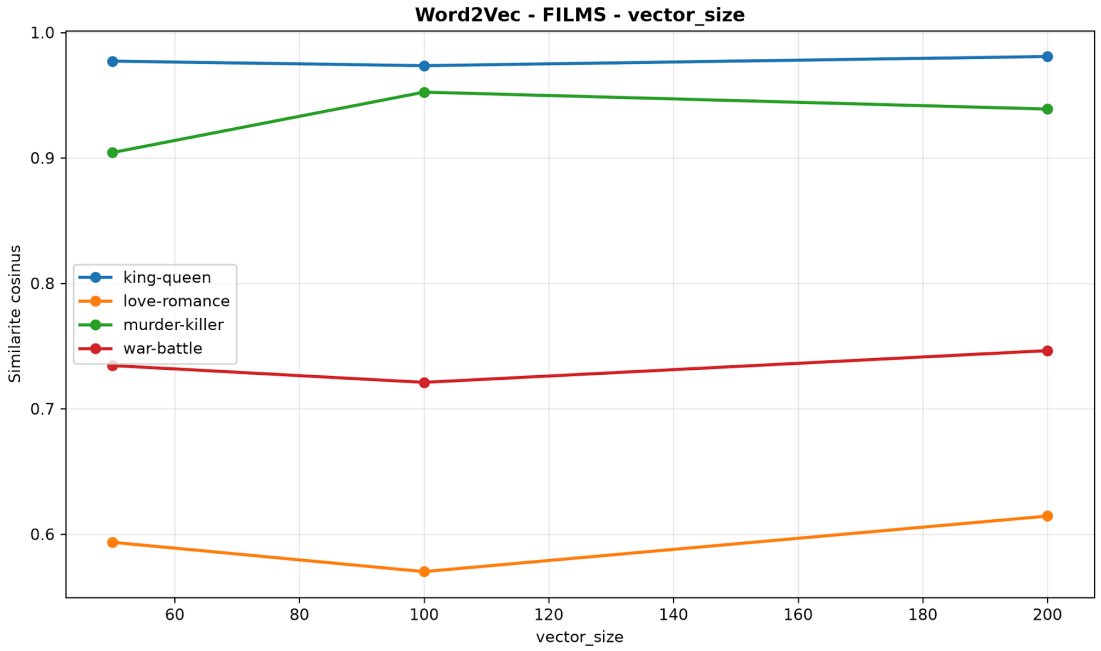


Figure 6 — Films : fenêtre contextuelle

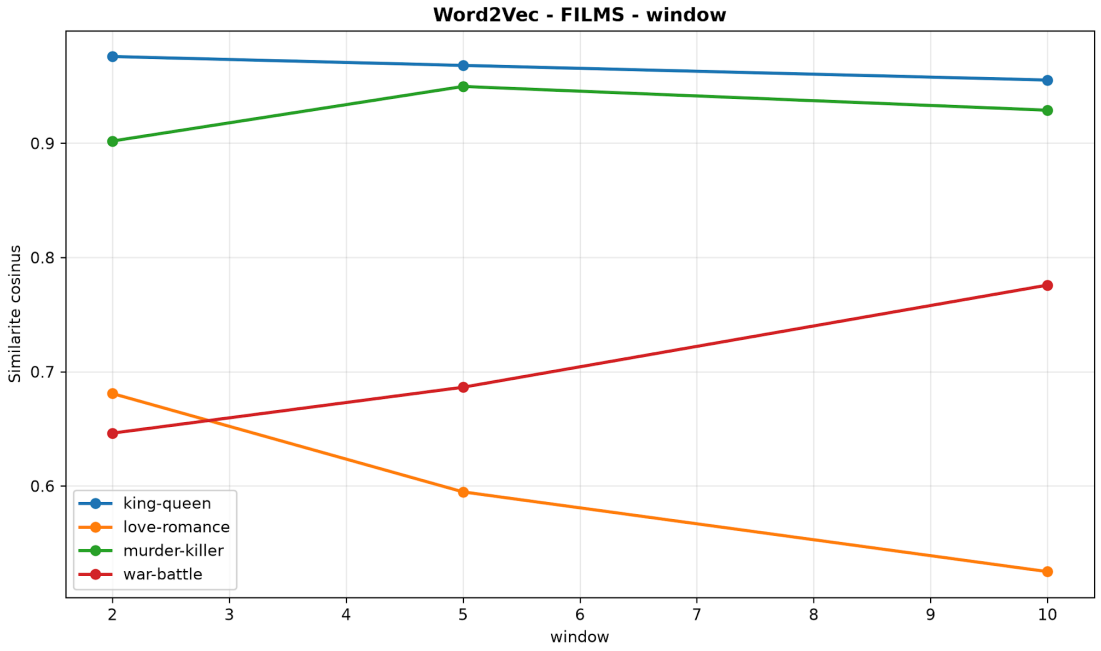


Figure 7 — Films : nombre d’époques

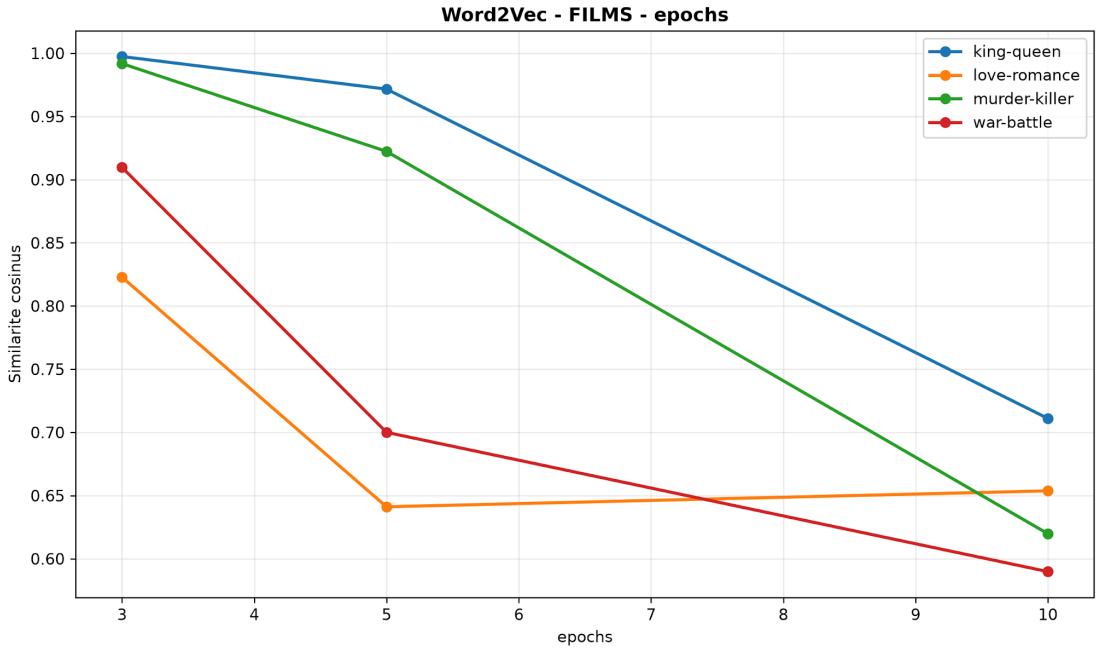


Figure 8 — Amazon : dimensions vectorielles

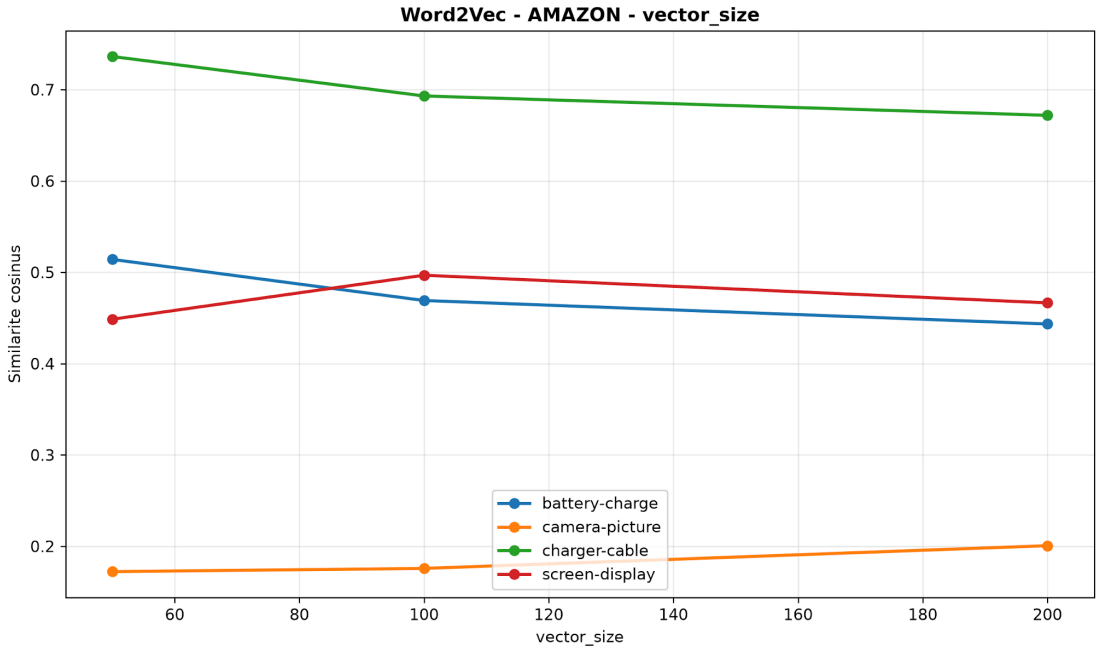


Figure 9 — Amazon : fenêtre contextuelle

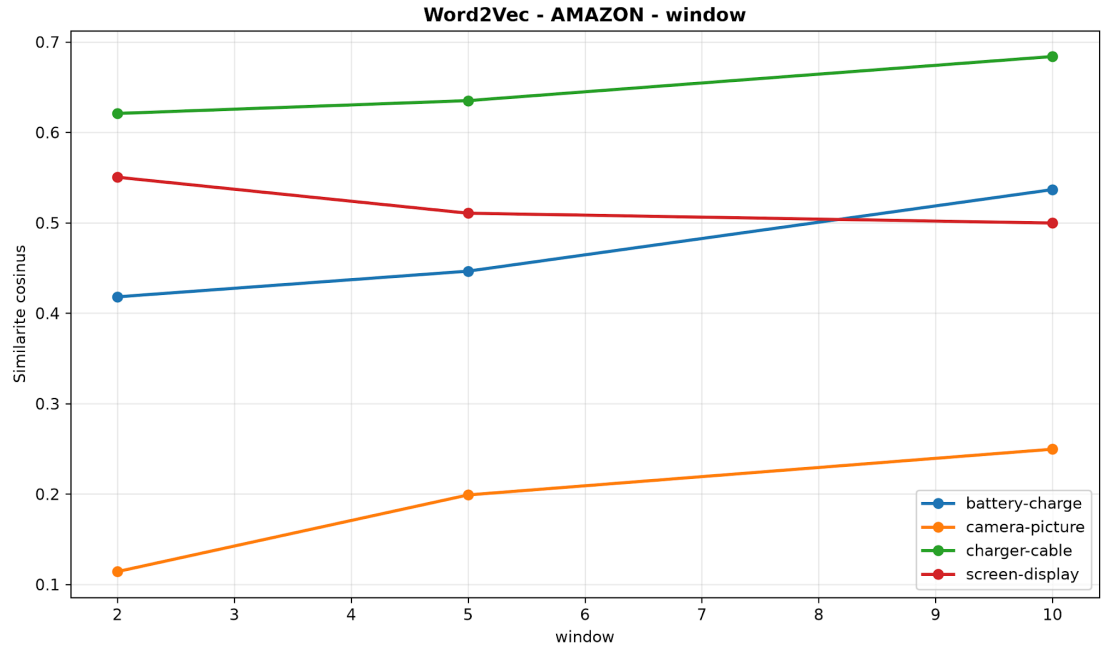
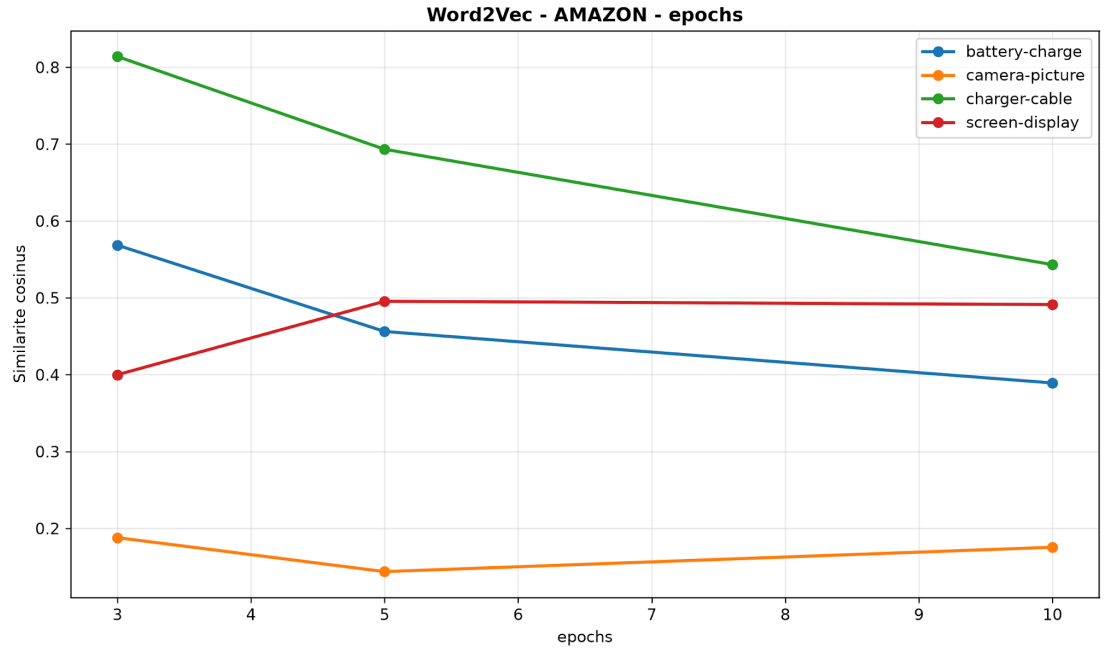


Figure 10 — Amazon : nombre d’époques



Les graphiques présentent les variations des similarités pour quatre paires cibles par corpus. Le réglage epochs=10 augmente le temps d'entraînement, mais ne garantit pas une augmentation de cosinus.
Dans les films, king–queen passe de 0,9974 à 3 époques à 0,7110 à 10 époques dans les expériences échantillonnées ; cette chute ne démontre pas une perte de qualité intrinsèque, puisqu'un cosinus excessivement élevé sur peu d'époques peut également traduire un espace encore peu discriminant.

Les modèles de référence réentraînés pour des tests distincts présentent de légères différences, car leur entraînement parallèle n'est pas nécessairement entièrement déterministe.

9. Limites et pistes d'amélioration

Premièrement, le corpus Amazon est susceptible de différer de l'archive Stanford explicitement désignée, et le document doit le reconnaître.
Deuxièmement, l'évaluation repose sur des voisins qualitatifs et des paires préalablement définies, non sur un benchmark externe annoté.
Troisièmement, les graphiques PCA ne constituent pas une métrique de qualité.
Quatrièmement, les tests d'hyperparamètres utilisent des sous-échantillons et une seule exécution par configuration ; ils ne doivent pas être comparés directement aux scores des modèles complets.
Enfin, l'absence de lemmatisation et de correction orthographique peut fragmenter le vocabulaire. On pourrait approfondir le TP avec plusieurs graines, des intervalles de variabilité et une évaluation sur une tâche aval (classification de textes, par exemple).

10. Conclusion

Ce travail répond au cœur de la consigne : deux corpus ont été chargés, nettoyés et tokenisés ; quatre modèles Word2Vec ont été entraînés avec Gensim ; des similarités, des voisinages lexicaux et des représentations PCA ont été analysés.

Les résultats quantifient une différence de coût importante entre CBOW et Skip-gram et révèlent des comportements dépendants du domaine. Les expériences complémentaires montrent que la dimension des vecteurs, la taille du contexte et la durée d'entraînement influencent les similarités observées.

L'interprétation reste prudente : une similarité cosinus plus élevée ne constitue pas, à elle seule, une preuve de meilleure qualité sémantique.

Références et code reproductible

Mikolov, T. et al. (2013), Efficient Estimation of Word Representations in Vector Space, arXiv:1301.3781.
Mikolov, T. et al. (2013), Distributed Representations of Words and Phrases and their Compositionality, NeurIPS.
Documentation de Gensim Word2Vec : https://radimrehurek.com/gensim/models/word2vec.html
The Movies Dataset : https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset
Stanford SNAP, archive citée dans la consigne : https://snap.stanford.edu/data/amazon/productGraph/categoryFiles/reviews_Cell_Phones_and_Accessories_5.json.gz
Support de TP : https://sally14.github.io/teaching/notebooks/TP4_dl_nlp.html
Dépôt du projet : https://github.com/NacPKZ/Word2Vec-NLP-Analysis

Corpus	Modèle	Dimensions	Fenêtre	Époques	Vocabulaire	Durée (s)
Films	CBOW	100	5	5	22 475	6,40
Films	Skip-gram	100	5	5	22 475	19,13
Amazon	CBOW	100	5	5	32 764	140,01
Amazon	Skip-gram	100	5	5	32 764	426,09

Corpus	Paire	CBOW	Skip-gram
Films	king–queen	0,7371	0,7588
Films	love–romance	0,6219	0,6554
Films	war–battle	0,6140	0,5072
Films	murder–killer	0,6750	0,5516
Films	family–mother	0,6227	0,4317
Amazon	phone–smartphone	0,4841	0,5594
Amazon	battery–charge	0,4246	0,5659
Amazon	screen–display	0,5338	0,6233
Amazon	camera–picture	0,2551	0,3976
Amazon	charger–cable	0,6141	0,6975